

Research and Development / AI Assignment

Parameter Estimation of a Parametric Curve using a Hybrid Differential Evolution and L-BFGS-B Optimization Framework

Submitted by:

Chanchlesh Suryawanshi

M.Tech, Data Science Engineering
National Institute of Technology, Jamshedpur

August 25, 2026

Contents

1	Introduction	2
2	Problem Formulation	2
3	Data Understanding	3
4	Alternative Optimization Methods Considered	3
4.1	Grid Search	3
4.2	Random Search	3
4.3	Gradient Descent	3
4.4	Genetic Algorithm	3
4.5	Bayesian Optimization	4
4.6	Summary	4
5	Proposed Hybrid Method: Differential Evolution + L-BFGS-B	4
5.1	Why Differential Evolution	4
5.2	Why L-BFGS-B for Refinement	4
5.3	Why This Combination	4
5.4	Mathematical Details of the Optimization Algorithms	5
5.4.1	Differential Evolution	5
5.4.2	L-BFGS-B Refinement	5
6	Optimization Procedure	5
7	Results	6
8	Discussion	6
9	Conclusion	7

Abstract

This report presents my approach to estimating the unknown parameters θ , M , and X of a given parametric curve from a noisy and unordered set of observed (x, y) points. The curve depends on these three parameters in a non-linear way, and the goal is to find the parameter values that make the curve pass as close as possible to the observed data, measured using the L1 (Manhattan) distance. To solve this, I used a two-stage hybrid optimization strategy: Differential Evolution (DE) for a global search over the parameter space, followed by L-BFGS-B for local refinement of the best solution found by DE. This combination allowed the search to first explore the space broadly and then fine-tune the result with higher precision. The final parameters obtained gave a low mean nearest-neighbour distance between the observed points and the fitted curve, indicating a good fit. This report explains the problem, the reasoning behind the chosen method, alternative methods that could also have worked, and the final results.

1 Introduction

Curve fitting and parameter estimation problems come up frequently in engineering and applied AI, especially when we are given a set of data points and a known mathematical model, but the exact parameters of that model are unknown. In this assignment, we were given a parametric curve defined by three unknown parameters, θ , M , and X , along with a dataset of observed points sampled from this curve (with noise and no particular ordering). The task was to recover the parameter values that best explain the observed data.

This is not a simple linear regression problem. The curve equations mix trigonometric and exponential terms, and the parameter θ affects the curve through a rotation-like transformation. Because of this non-linearity, and because we do not know which observed point corresponds to which value of the curve parameter t , a direct closed-form solution (OLS) is not possible. Instead, this becomes an optimization problem: we need to search over the space of possible (θ, M, X) values and find the combination that minimizes the distance between the observed points and the generated curve.

In this report, I have explained the mathematical formulation of the problem, described the data that I was given, discussed a few optimization methods that could reasonably be used to solve it, and then explained in detail the method I actually implemented: a combination of Differential Evolution and L-BFGS-B.

2 Problem Formulation

The parametric curve is defined as:

$$x(t) = t \cos(\theta) - e^{M|t|} \sin(0.3t) \sin(\theta) + X \quad (1)$$

$$y(t) = 42 + t \sin(\theta) + e^{M|t|} \sin(0.3t) \cos(\theta) \quad (2)$$

where t is the curve parameter, restricted to the range $6 < t < 60$, and θ , M , X are the unknown parameters we want to estimate, subject to the following bounds:

- $0^\circ < \theta < 50^\circ$
- $-0.05 < M < 0.05$
- $0 < X < 100$

Since the observed data points are not labelled with their corresponding t value, and the ordering of the points is not known, we cannot directly match an observed point to a specific point on the curve. To get around this, we treat the curve as a densely sampled set of points (generated by evaluating $x(t)$ and $y(t)$ at many values of t), and for every observed point we find its nearest point on this dense curve. The objective function to minimize is the mean L1 distance between each observed point and its nearest curve point [1]:

$$\mathcal{L}(\theta, M, X) = \frac{1}{N} \sum_{i=1}^N \min_j \left(|x_i^{obs} - x_j^{curve}| + |y_i^{obs} - y_j^{curve}| \right) \quad (3)$$

where N is the number of observed points, and j ranges over all sampled points on the candidate curve generated using the current (θ, M, X) .

This makes the problem a three-variable, bounded, non-convex, black-box optimization problem. It is "black-box" in the sense that the objective function does not have a simple closed-form gradient with respect to θ , M , X , because the nearest-neighbour matching step changes discontinuously as the parameters change.

3 Data Understanding

The dataset provided contains a set of observed (x, y) coordinate pairs. Before running any optimization, I inspected the data to check its size, range, and general shape. A few important of the observations are:

- The points do not follow any particular order, so they cannot be simply plotted in sequence to reveal the curve. They need to be treated as an unordered point cloud.
- The data contains some scatter around the true underlying curve, which is expected since real or simulated measurements are rarely perfectly clean.
- Plotting the raw points gives a rough visual idea of the curve's shape and location, which is useful for sanity-checking the final fitted parameters later on.

Understanding the data before optimization is a useful step because it helps confirm, after the fact, whether the recovered curve actually passes through the cloud of points in a visually sensible way, and not just according to the numerical objective value.

4 Alternative Optimization Methods Considered

Before finalizing the approach, I considered several optimization methods that are commonly used for problems like this. It is worth mentioning that because this problem has only three unknown variables (θ , M , X), the search space is relatively small and low-dimensional compared to many real-world optimization problems. This means that quite a few different optimization strategies are capable of finding a good solution here, not just the one I finally chose. The goal of this assignment was to build a working and reliable parameter estimation pipeline, not to benchmark or compare every possible optimization algorithm against each other.

4.1 Grid Search

Our search space is continuous. Grid search evaluates the objective at every combination of parameter values on a fixed grid. With only three bounded parameters, a fine enough grid could cover the space and locate a good region. But when the search space increases then using grid search is not good approach because it exhaustively searches the entire search space which is computationally not efficient. Its main appeal is simplicity and the fact that it does not depend on randomness.

4.2 Random Search

Random search samples parameter combinations randomly within the bounds instead of using a fixed grid. For low-dimensional problems like this one, it often performs comparably to grid search, and sometimes better, since it does not waste evaluations on a rigid grid.

4.3 Gradient Descent

Gradient descent moves iteratively towards a minimum using the objective's gradient. Since the objective here can be approximated numerically through finite differences even without an exact analytical gradient, a gradient-based method could still be applied, especially to refine a starting guess. But the main problem here using gradient descent is that the modulus is used in the L1 (Manhattan) distance, which is not differential at some points.

4.4 Genetic Algorithm

A Genetic Algorithm (GA) is a population-based search method inspired by natural selection, using selection, crossover, and mutation. GAs are well suited to non-convex, discrete objective functions, since they do not need gradient information, but our search space is continuous [2], [3].

4.5 Bayesian Optimization

Bayesian Optimization builds a probabilistic surrogate model of the objective and uses it to decide where to sample next [4]. It is useful when each evaluation is expensive, and it tends to need fewer evaluations to find a good solution compared to purely random search.

4.6 Summary

All of these methods could realistically solve this problem, mainly because the search space is small (three variables) with well-defined bounds. My conclusions on suitability are based on general knowledge from optimization literature and the theoretical properties of each method, not on a direct experimental comparison where every one of these approaches was implemented and tuned on this dataset. Given the scope of this assignment, I focused on implementing one strong, well-justified approach rather than a full comparative study. The point is that all of the solutions that were discussed in the previous section can solve this problem, but my main goal while solving this problem was to give a solution that can work more efficiently compared to other solutions that we can possibly try.

5 Proposed Hybrid Method: Differential Evolution + L-BFGS-B

For this assignment, I chose to implement a two-stage hybrid optimization method that combines Differential Evolution (DE) with L-BFGS-B.

5.1 Why Differential Evolution

Differential Evolution is a well-established, population-based global optimization algorithm for continuous search spaces [5]. It works by maintaining a population of candidate solutions and generating new candidates by combining the differences between existing population members, then keeping whichever candidate performs better. It does not require gradient information, which fits well with our objective function, since the nearest-neighbour distance calculation is not smooth in the traditional sense (small parameter changes can cause the nearest-neighbour matching to jump between different curve points).

I chose DE specifically because it has a long history of use in continuous global optimization problems and is well documented in the optimization literature [6]. It also handles bounded search spaces naturally, which matches the constraints given in this assignment ($0^\circ < \theta < 50^\circ$, $-0.05 < M < 0.05$, $0 < X < 100$).

5.2 Why L-BFGS-B for Refinement

After DE finds a good candidate solution, I refine it further using L-BFGS-B. L-BFGS-B is a quasi-Newton method that supports box (bound) constraints directly, without any extra reformulation of the problem [7]. Once DE has narrowed the search to a good region, the objective tends to behave more smoothly there, and L-BFGS-B can use numerically estimated gradients to converge more precisely to a local minimum [8].

I did not use Levenberg-Marquardt (LM) for this stage, even though LM is a common choice for curve fitting. LM is designed for least-squares (L2 norm) problems where each observed point has a fixed, differentiable residual with respect to the parameters. Here, the objective is based on L1 (Manhattan) distance to the *nearest* curve point, and this nearest-point assignment can change abruptly as the parameters change, which breaks LM's core assumption. LM is also naturally unconstrained, so enforcing the given bounds would need extra work. L-BFGS-B, in contrast, supports bounds directly and treats the objective as a general scalar function, which fits this refinement stage well.

5.3 Why This Combination

The reasoning behind combining these two methods is straightforward: DE is good at global exploration but can be relatively slow to converge to a very precise answer, while L-BFGS-B is fast and precise but only works well once it starts close to a good solution (a local method). Using DE first gives us a solid starting point that is unlikely to be stuck in a bad local minimum, and then L-BFGS-B polishes that solution to get more accurate final parameter values. This gives us both global exploration and local exploitation in one pipeline.

5.4 Mathematical Details of the Optimization Algorithms

5.4.1 Differential Evolution

Differential Evolution (DE) is a population-based global optimization algorithm. Let

$$\mathbf{x}_i^{(g)} = [\theta_i^{(g)}, M_i^{(g)}, X_i^{(g)}]$$

denote the i -th candidate solution in generation g . For each candidate, DE creates a mutant vector using three randomly selected population members:

$$\mathbf{v}_i^{(g)} = \mathbf{x}_{r_1}^{(g)} + F \left(\mathbf{x}_{r_2}^{(g)} - \mathbf{x}_{r_3}^{(g)} \right),$$

where F is the mutation factor and r_1, r_2, r_3 are distinct indices. The mutant vector is combined with the current candidate through crossover to generate a trial vector:

$$\mathbf{u}_i^{(g)}$$

The trial vector replaces the original candidate if it achieves a lower objective value:

$$\mathbf{x}_i^{(g+1)} = \begin{cases} \mathbf{u}_i^{(g)}, & \text{if } \mathcal{L}(\mathbf{u}_i^{(g)}) < \mathcal{L}(\mathbf{x}_i^{(g)}), \\ \mathbf{x}_i^{(g)}, & \text{otherwise.} \end{cases}$$

This process is repeated for multiple generations until convergence or a stopping criterion is reached.

5.4.2 L-BFGS-B Refinement

After Differential Evolution identifies a promising region of the search space, the best solution found by DE is used as the initial point for L-BFGS-B. The method seeks a parameter update

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \alpha_k \mathbf{p}_k,$$

where α_k is a step size obtained through line search and $\mathbf{p}_k$ is the search direction. Unlike classical Newton's method, which requires the full Hessian matrix,

$$H = \nabla^2 \mathcal{L}(\mathbf{x}),$$

L-BFGS-B constructs a low-memory approximation using information from recent iterations. The search direction is computed as

$$\mathbf{p}_k = -B_k^{-1} \nabla \mathcal{L}(\mathbf{x}_k),$$

where B_k^{-1} is the approximate inverse Hessian matrix. The algorithm additionally enforces the parameter bounds

$$\begin{aligned} 0^\circ &\leq \theta \leq 50^\circ, \\ -0.05 &\leq M \leq 0.05, \\ 0 &\leq X \leq 100, \end{aligned}$$

through projection onto the feasible region. This makes L-BFGS-B particularly suitable for bounded optimization problems such as this.

6 Optimization Procedure

The implementation follows these steps:

1. **Load the data.** The observed (x, y) points are read from the CSV file.

2. **Define the objective function.** For any given (θ, M, X) , the curve is sampled densely over $t \in [6, 60]$ (20,000 points during optimization), and a KD-tree is built over these sampled curve points. For each observed point, the nearest curve point is found using an L1 distance query, and the mean of these distances across all observed points gives the objective value.
3. **Stage 1 – Differential Evolution.** DE is run over the bounded search space $\theta \in [0^\circ, 50^\circ]$, $M \in [-0.05, 0.05]$, $X \in [0, 100]$, using the `best1bin` strategy with a population size of 40 and up to 2000 generations. This stage explores the space broadly and returns the best candidate solution found.
4. **Stage 2 – L-BFGS-B Refinement.** The best solution from DE is used as the starting point for L-BFGS-B, which is again run with the same parameter bounds. This stage fine-tunes the parameters to reduce the objective value further.
5. **Final evaluation.** The refined parameters are used to regenerate the curve at a higher resolution (50,000 points), and the final error metrics (mean, median, and maximum L1 and Euclidean distances, RMSE, MAE, and standard deviation of residuals) are computed.
6. **Visualization.** Several plots are generated to visually confirm the fit: the raw observed points, the recovered curve alone, an overlay of both, a zoomed-in view, residual histograms, a residual magnitude scatter plot, and a comparison between the DE-only solution and the final refined solution.

This two-stage pipeline was implemented in Python, using SciPy’s `differential_evolution` and `minimize` (with the L-BFGS-B method) functions from `scipy.optimize`, along with `cKDTree` from `scipy.spatial` for fast nearest-neighbour lookups [9].

7 Results

The two-stage optimization produced the following parameter estimates:

- $\theta = 29.999988^\circ$
- $M = 0.030001$
- $X = 55.000003$

These values are extremely close to the round numbers $(30^\circ, 0.03, 55)$, indicating that the optimization procedure successfully recovered the underlying parameters rather than converging to a spurious local minima.

The Differential Evolution (DE) stage located a promising region of the search space and achieved an fitness value of 0.013945. However, DE reached its maximum iteration limit (30 iterations and 930 function evaluations) before satisfying its convergence criterion. The subsequent L-BFGS-B refinement stage further reduced the objective value to 0.006914, representing a **50.42% improvement**. This demonstrates that the local optimization stage contributed significantly to the final solution quality rather than serving merely as a formality.

Several error metrics were computed to evaluate the fit between the optimized curve and the observed data. The mean L1 distance was only 0.000428, while the mean Euclidean distance was 0.000341. The Root Mean Square Error (RMSE) and Mean Absolute Error (MAE) were 0.000391 and 0.000341, respectively. Furthermore, the standard deviation of the residuals was 0.000192, indicating that the errors were tightly clustered around the mean rather than being widely dispersed. The maximum Euclidean distance observed was only 0.000943, suggesting the absence of significant outliers.

The optimization was executed on the standard Google Colab runtime using its default computational resources. Differential Evolution completed in approximately 2.24 seconds, while the subsequent L-BFGS-B refinement required an additional 0.56 seconds. The total execution time was therefore approximately 2.80 seconds. These results indicate that the proposed hybrid optimization framework achieves high solution accuracy while maintaining a low computational cost.

8 Discussion

The optimized parameters ($\theta \approx 30^\circ$, $M \approx 0.03$, $X \approx 55$) generated a curve that closely matched the observed data points. The consistently small residuals across all evaluation metrics demonstrate that the fitted curve not only approximated the data globally but also tracked individual observations with high

precision.

One notable observation is the contribution of the L-BFGS-B refinement stage. Although Differential Evolution had already identified a solution near the optimum, the final parameter values obtained after refinement were noticeably more accurate. The DE solution ($\theta = 30.02^\circ$, $M = 0.0301$, $X = 55.02$) was close to the final result, yet the refinement stage reduced the objective value by more than half. This highlights a common characteristic of Differential Evolution: it is highly effective at locating the correct region of the search space but is generally less efficient at achieving very high numerical precision.

The local refinement stage also demonstrated superior efficiency once the search was confined to the vicinity of the optimum. L-BFGS-B converged successfully using only 300 function evaluations, whereas Differential Evolution required 930 evaluations without reaching its formal convergence criterion. This illustrates the advantage of combining a global optimization method with a gradient-based local optimizer.

Overall, the results validate the choice of a hybrid DE + L-BFGS-B optimization strategy. Differential Evolution effectively handled the global, potentially non-convex search problem, while L-BFGS-B efficiently refined the solution to a high degree of accuracy. Although alternative optimization approaches such as gradient descent, Bayesian optimization, or genetic algorithms could potentially solve this three-parameter bounded optimization problem, the chosen hybrid approach achieved excellent accuracy with a very modest computational cost. The recovered parameters closely matched the expected values, and the resulting error metrics confirmed the quality of the final fit.

9 Conclusion

In this assignment, I estimated the unknown parameters θ , M , and X of a given parametric curve using a hybrid optimization approach that combines Differential Evolution for global search with L-BFGS-B for local refinement. The problem was formulated as minimizing the mean L1 distance between observed data points and their nearest points on a candidate curve. I also discussed several alternative optimization methods, including Grid Search, Random Search, Gradient Descent, Genetic Algorithms, and Bayesian Optimization, all of which could reasonably be applied to this problem given its low dimensionality. The final hybrid method produced a curve that fit the observed data well, both numerically (through low mean and median residual distances) and visually (through the overlay and zoomed plots). This exercise reinforced how important it is to match the choice of optimization method to the specific properties of the objective function, such as boundedness and non-smoothness, rather than picking a method purely based on popularity.

References

- [1] T. Cover and P. Hart, "Nearest neighbor pattern classification," *IEEE transactions on information theory*, vol. 13, no. 1, pp. 21–27, 1967.
- [2] D. Goldberg, *Genetic Algorithms in Search, Optimization, and Machine Learning* (Addison Wesley series in artificial intelligence). Addison-Wesley, 1989, ISBN: 9780201157673. [Online]. Available: https://books.google.co.in/books?id=3_RQAAAAMAAJ
- [3] J. Holland, *Adaptation in Natural and Artificial Systems: An Introductory Analysis with Applications to Biology, Control, and Artificial Intelligence*. University of Michigan Press, 1975, ISBN: 9780472084609. [Online]. Available: <https://books.google.co.in/books?id=JE5RAAAAMAAJ>
- [4] J. Snoek, H. Larochelle, and R. Adams, "Practical bayesian optimization of machine learning algorithms," in *Advances in Neural Information Processing Systems*, F. Pereira, C. Burges, L. Bottou, and K. Weinberger, Eds., vol. 25, Curran Associates, Inc., 2012, pp. 2951–2959. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2012/file/05311655a15b75fab86956663e1819cd-Paper.pdf
- [5] R. Storn and K. Price, "Differential evolution—a simple and efficient heuristic for global optimization over continuous spaces," *Journal of global optimization*, vol. 11, no. 4, pp. 341–359, 1997.
- [6] K. V. Price, R. M. Storn, and J. A. Lampinen, *Differential evolution: a practical approach to global optimization*. Springer, 2005.

- [7] R. H. Byrd, P. Lu, J. Nocedal, and C. Zhu, “A limited memory algorithm for bound constrained optimization,” *SIAM Journal on scientific computing*, vol. 16, no. 5, pp. 1190–1208, 1995.
- [8] J. Nocedal and S. J. Wright, *Numerical optimization*. Springer, 2006.
- [9] P. Virtanen et al., “Scipy 1.0: Fundamental algorithms for scientific computing in python,” *Nature methods*, vol. 17, no. 3, pp. 261–272, 2020.